Aplicații AA
Tips and Tricks

radu.iacob23@gmail.com

January 2021

Contents

1 Decidabilitate **1**

2 Algoritmi Nedeterminiști **2**
2.1 Tips and Tricks 2

3 Teorema Master **3**
3.1 Tip and Tricks 3

4 Inducție Structurală **4**
4.1 Tips and Tricks 8

1 Decidabilitate

1. Fie problema Q, ce primește un program la intrare P, o mulțime finită A și verifică dacă pentru orice x din A, P(x) se termină. Demonstrați că problema Q este decidabilă, semidecidabilă sau nedecidabilă.

   (a) Primul pas: Înțelegem ce fel de problema este Q.
   Tipul intrării: **un program (P)**, **o mulțime finită (A)**
   Tipul ieșirii: **bool** (problema de decizie)
   Header-ul unei solutii pentru problema Q ar fi: $AlgoQ : P, A \rightarrow Bool$

   (b) Pentru a demonstra că o problemă este decidabilă trebuie să identificăm o soluție care să producă întotdeauna rezultatul corect in timp finit. Observăm însă că enunțul problemei Q este o forma mai generala a problemei opririi (Halting Problem), despre care știm că este semidecidabilă.

   (c) Dem. că există un program care rezolvă Q și se termină în timp finit **dacă problema are soluție**.

```
function SolveQ(P, A(Set))
    for x in A do
        P(x)
    end for
    Return true
end function
```

(d) Dem. că problema este nedecidabilă (în particular, semidecidabilă) deoarece putem reduce problema opririi la Q.
$HALT <_T Q$

i. Definim funcția de transformare F (Turing calculabilă) care convertește orice instanță a unei probleme de tip Halt într-o instanță specială a problemei Q.

ii. $F : Input_{HALT} \Rightarrow Input_Q$
$F : (P, x) \Rightarrow (P, A)$
Soluție: Alegem $A = \{x\}$ (mulțimea care conține doar intrarea x) și lăsăm P nemodificat.

iii. Dacă am avea un algoritm care să rezolve în **timp finit** acestă instanță a problemei Q, am putea rezolva **orice** problema de tip Halt.

iv. Demonstrăm $Halt(P, x) = 1 \rightarrow Q(P, x) = 1$.
Dacă programul P se termină pe intrarea x, atunci și SolveQ() se va termina cu succes.

v. Demonstrăm $Q(P, x) = 1 \rightarrow Halt(P, x) = 1$.
SolveQ() nu se poate termina cu succes decât dacă P(x) se termină.

2 Algoritmi Nedeterminiști

1. Pentru startup-ul vostru, FaceAlgo, doriți să atribuiți proiectele angajaților știind că fiecare angajat poate lucra la un singur proiect la un moment dat. Se dau setul P de proiecte și E de angajați, iar pentru fiecare proiect $p \in P$ o submulțime $E_p \subseteq E$ de angajați care trebuie alocați proiectului p pentru a-l finaliza în timp util. Determinați dacă se pot aloca angajații pe proiecte astfel încât să fie finalizate (cel puțin) k proiecte.

2. Fie o mutime S de numere reale și doi întregi pozitivi, K și L. Este posibilă partiționarea lui S în K submulțimi disjuncte astfel încât fiecare submulțime să conțină $2L$ elemente, iar suma elementelor din fiecare submulțime să fie zero?

2.1 Tips and Tricks

- O problemă de decizie aparține NP dacă se poate verifica, în timp polinomial, cu un algoritm determinist, dacă o soluție respectă toate constrângerile problemei.

- Alternativ, o problemă aparține NP dacă o putem rezolva folosind un algoritm nedeterminist polinomial.
- Soluția unei probleme constă din V variabile, instanțiate astfel încât să satisfacă C constrângeri. Primul pas este să identificăm V, respectiv C.
- Un pattern general de rezolvare ar fi:

```
function SOLVE(X)
        // Generare
    for var in V do
        var = choice(domeniul de valori a lui var)
    end for
        // Validare
        for constraint c in C do
            for relevant var v in V do
                if v doesn't respect c then fail.
                end if
            end for
        end for
        success
        end function
```

- Atenție! Operația **choice** primește la intrare **o mulțime finită** și întoarce un element din respectiva mulțime. Exemple valide: mulțimea nodurilor dintr-un graf, mulțimea codurilor ASCII, etc.
- Ambele etape (generare, validare) trebuie implementate în timp **polinomial**.

3 Teorema Master

- $T(n) = 32 * T(\frac{n}{2}) + log_2(n^n)^5$
- $T(n) = 3 * T(\frac{n}{8}) + 2021 * n^{\frac{1}{3}}$
  - Trebuie să compar $n^{log_8(3)}$ cu $f(n)$.
  - Hint: Pot să scriu $\frac{1}{3}$ ca un logaritm de bază 8.

3.1 Tip and Tricks

- Asigurați-vă că se îndeplinesc constrângerile pentru a putea aplica Teorema Master: a, b **constante**, $a \geqslant 1$, $b > 1$, $f(n)$ asimptotic pozitivă.
- Nu uitați de ϵ din formulă! Pentru a aplica Teorema Master $f(n)$ aceasta trebuie să fie **polinomial** mai mică (cazul 1) sau mai mare (cazul 3) decât $n^{log_b(a)}$.

- Expresia $\frac{n}{log(n)}$ **nu** este polinomial mai mică decât n.

- **Recapitulați proprietățile logaritmilor**.
- Puteți folosi metoda substituției pentru a vă verifica.
- **How to prepare**:
  - Seminar 3
  - Math Cheatsheet
  - Test 1 2018 (rezolvări)
  - Test 1 2019 (rezolvări)

4 Inducție Structurală

List 1: Constructori

- $[] :\rightarrow ListN$
- $[x] : N \rightarrow ListN$
- $(x : xs) : N \rightarrow (ListN \backslash []) \rightarrow ListN$

List 2: $head : (ListN \backslash []) \rightarrow N$

H1: $head([x]) = x$

H2: $head(x : xs) = x$

List 3: $last : (ListN \backslash []) \rightarrow N$

L1: $last([x]) = x$

L2: $last(x : xs) = last(xs)$

List 4: $sorted : ListN \rightarrow Bool$

S1: $sorted([]) = True$

S2: $sorted([x]) = True$

S3: $sorted(x : xs) = x \leqslant head(xs) \wedge sorted(xs)$

List 5: $(++) : ListN \to ListN \to ListN$

APP1: $[] + +B = B$

APP2: $(x : xs) + +B = x : (xs + +B)$

List 6: $filter : ListN \to Fun1 \to ListN$

F1: $filter([], f) = []$

F2:

$$filter([x], f) = \begin{cases} [x] & f(x) == True \\ [] & altfel \end{cases}$$

F3:

$$filter(x : xs, f) = \begin{cases} x : filter(xs, f) & f(x) == True \\ filter(xs, f) & altfel \end{cases}$$

Prototipul funcțiilor Fun1 este: $Fun1 : N \to Bool$

List 7: $size : ListN \to N$

SZ1: $size([]) = 0$

SZ2: $size([x]) = 1$

SZ3: $size(x : xs) = 1 + size(xs)$

List 8: $member : ListN \to N \to Bool$

M1: $member([], e) = False$

M2: $member([x], e) = (x == e)$

M3: $member(x : xs, e) = (x == e) \vee member(xs, e)$

De demonstrat P 1

$A \neq [] \implies sorted(A + +[a]) = (sorted(A) \wedge a \geqslant last(A))$
Presupunem cunoscut: $[x] + +B = x : B$.

De demonstrat P2

$member(A, e) \implies size(A) > size(filter(A, diff(e)))$
unde $diff(e) = \lambda x : e \neq x$
- o funcție care primește ca argument un element x și verifică dacă este diferit de e

Demonstrație P1.

- **Cazul de bază**: $A == []$

  $A \neq [] = False$, $False \implies True$.

- **Cazul de bază**: $A == [x]$

  Ne propunem să demonstrăm:
  $sorted([x] + +[a]) = sorted([x]) \wedge a \geqslant last([x])$.

  $LHS = sorted([x] + +[a]) = sorted(x : [a])$
  $=^{S3} x \leqslant head([a]) \wedge sorted([a])$
  $=^{H1} x \leqslant a \wedge sorted([a])$.

  Din S2 știm că $sorted([a]) == sorted([x]) == True \implies LHS = RHS$.

- **Ipoteza inductivă - P(xs)**:
  $xs \neq [] \implies sorted(xs + +[a]) = sorted(xs) \wedge a \geqslant last(xs)$.

- **Pasul de inducție**: $P(xs) \implies P(x : xs)$.

  Ne propunem să demonstrăm:
  $(x : xs) \neq [] \implies sorted((x : xs) + +[a]) = sorted(x : xs) \wedge a \geqslant last(x : xs)$.

  $(x : xs) \neq [] = True$

  $LHS = sorted((x : xs) + +[a]) =^{APP2} sorted(x : (xs + +[a]))$
  $=^{S3} x \leqslant head(xs + +[a]) \wedge sorted(xs + +[a])$
  $=^{II} x \leqslant head(xs + +[a]) \wedge sorted(xs) \wedge a \geqslant last(xs)$.

  $RHS = sorted(x : xs) \wedge a \geqslant last(x : xs)$
  $=^{S3} x \leqslant head(xs) \wedge sorted(xs) \wedge a \geqslant last(xs)$.

  Mai avem de demonstrat: PA(xs):
  $xs \neq [] \implies head(xs + +[a]) == head(xs)$.
  Dacă o aplicăm direct obținem: $LHS = RHS$.

  Această proprietate adițională poate fi demonstrată folosind tot inducție structurală.

- **Cazul de bază** $xs == []$.
  $xs \neq [] == False, False \implies True.$

- **Cazul de bază** $xs = [x]$
  Ne propunem să demonstrăm:
  $xs \neq [] \implies head(xs + +[a]) == head(xs)$

  $xs \neq [] == True.$
  $LHS = head([x] + +[a]) = head(x : [a])$
  $=^{H1} x.$
  $RHS = head([x]) =^{H1} x \implies LHS = RHS.$

- **Ipoteza inductivă**
  $P(xs) : xs \neq [] \implies head(xs + +[a]) == head(xs).$

- **Pasul de inducție** $P(xs) \implies P(x : xs).$
  Ne propunem să demonstrăm că:
  $(x : xs) \neq [] \implies head((x : xs) + +[a]) == head(x : xs).$

  $(x : xs) \neq [] == True.$
  $LHS = head((x : xs) + +[a]) =^{APP2} head(x : (xs + +[a])) =^{H2} x.$
  $RHS = head(x : xs) =^{H2} x \implies LHS = RHS.$

Demonstrație P2.

- **Cazul de bază**: A == []
  $member([], e) =^{M1} False, False \implies True.$
- **Cazul de bază:** A == [x]
  Ne propunem să demonstrăm:
  $member([x], e) \implies size([x]) > size(filter([x], diff(e)))s.$

  $member(x, e) =^{M2} (x == e).$

  $LHS = size([x]) =^{SZ2} 1.$

  $RHS = size(filter([x], diff(e))) =^{F2} size(x \neq e?[x] : []) = size([])$
  $=^{SZ1} 0 \implies LHS > RHS.$
- **Ipoteza inductivă**:
  $P(xs) : member(xs, e) \implies size(xs) > size(filter(xs, diff(e))).$

– **Pasul de inducție**: $P(xs) \implies P(x : xs)$.
Ne propunem să demonstrăm:
$member(x : xs, e) \implies size(x : xs) > size(filter(x : xs, diff(e)))$.

$member(x : xs, e) = (x == e) \vee member(x, e)$.

$LHS =^{SZ1} = 1 + size(xs)$.

$$RHS = \begin{cases} 1 + size(filter(xs, diff(e))), & x \neq e \\ size(filter(xs, diff(e)))), & altfel \end{cases}$$

Dacă presupunem $member(x : xs, e)$ adevărat, trebuie să analizăm toate posibilitățile.

1. $(x == e)$
$RHS = size(filter(xs, diff(e)))$.
Pentru a demonstra $LHS > RHS$ avem nevoie de o proprietate adițională:
$PA(xs) : size(xs) \geqslant size(filter(xs, f))$.

2. $member(xs, e) \wedge (x \neq e)$.

   $RHS = 1 + size(filter(xs, diff(e)))$.

   $\implies {}^{II} LHS > RHS$.

3. $(x == e) \wedge member(xs, e)$ (analog cu primele două)

– Mai avem de demonstrat:
$PA(xs) : size(xs) \geqslant size(filter(xs, f))$.

4.1 Tips and Tricks

- Pașii de urmat pentru a demonstra proprietatea folosind inducție structurală:

  1. **Cazul de bază**: Demonstrăm proprietatea pentru toți constructorii nulari și externi.
  2. Alegem variabila pentru care vom face inducție.
  3. **Ipoteza inductivă**: Presupunem că, pentru fiecare constructor intern, proprietatea este respectată de **parametrii** de tip T.

4. **Pasul de inducție**: Folosind ipoteza inductivă, arătăm că proprietatea este adevărată pentru valorile de tip T **întoarse** de fiecare constructor intern.

- La examen, puteți introduce notații noi pentru a scrie mai puțin ("Notăm sorted() cu s()"). Menționați noile notații la începutul demonstrației.
- Nu săriți pași! Dacă o proprietate necesară este 'evidentă' probabil este necesară o proprietate adițională care trebuie demonstrată prin inducție.
- De asemenea, o proprietate adițională este necesară atunci când nu mai putem înainta in demonstrație folosind doar axiomele/proprietățile cunoscute.
- Precizați clar axiomele folosite.
- Dacă sunt posibile mai multe valori de adevăr, **analizați separat pentru toate cazurile**.